TRAFFIC ACCIDENT SEVERITY PREDICTION

Project Id: 20

Name : Priyani Singh (Team Leader)

Class Roll Number: 59

University Roll Number : 2028806

Section and Group: P and G2

Name : Suditi

Class Roll Number: 70

University Roll Number : 2028868

Section and Group: P and G2

TABLE OF CONTENTS

1. **Problem Description..... 3**
   - 1.1. **Dataset Description..... 3**
   - 1.2. **Flow diagram..... 4**

2. **Modules for Project Implementation……..……5**
   - 2.1. **Modules Used.....5**
   - 2.2. **Platform Used.....6**

3. **Machine Learning model/ Searching Technique used..... 7**
4. **Evaluation metrics used.....8**
5. **Results and Visualizations..... 9**
6. **Conclusion and Future Scope..... 11**
7. **References..... 12**

1. Problem Description

The increasing number of road traffic accidents has become a major concern worldwide, leading to loss of lives, serious injuries, and economic damage. Predicting the severity of road accidents can help traffic authorities and healthcare services take preventive measures and improve road safety. In this project, machine learning techniques are used to analyze accident-related data such as driver details, weather conditions, road conditions, vehicle information, and number of casualties to predict accident severity. The dataset contains multiple factors influencing accidents, and different classification algorithms are applied to identify the most accurate predictive model. The main objective of this project is to develop an efficient accident severity prediction system that can assist in reducing accident risks and improving transportation safety.

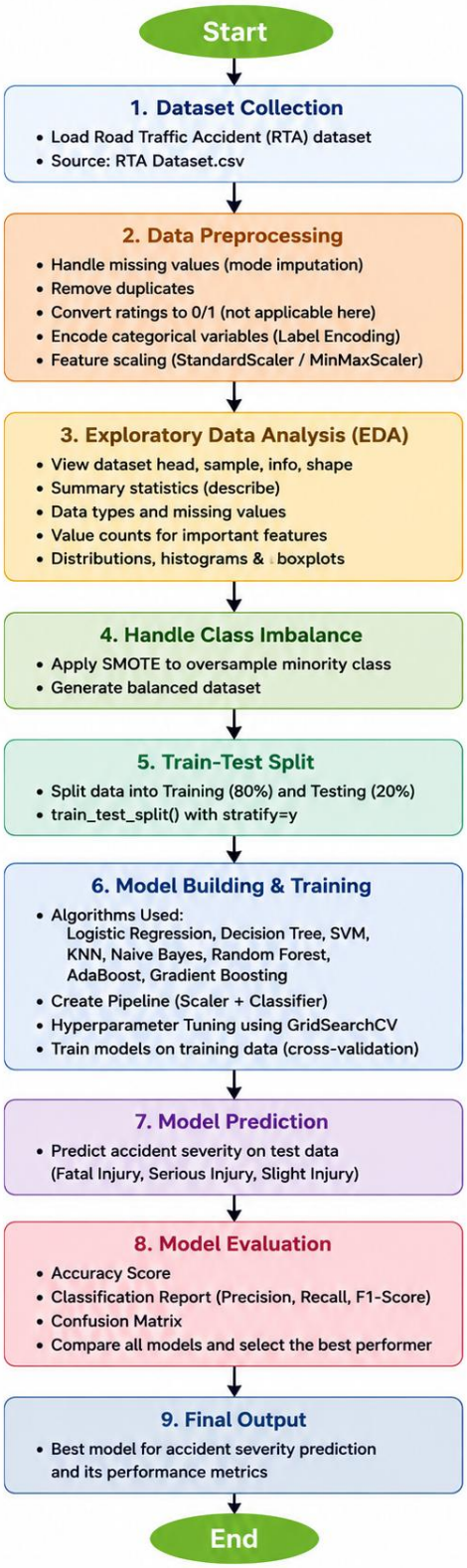
1.1 Dataset Description

The dataset plays an important role in developing and evaluating the accident severity prediction model. It contains detailed information about road traffic accidents, including driver details, vehicle information, environmental conditions, and accident outcomes. The dataset structure helps in understanding the relationship between different factors affecting road accidents and supports effective data analysis and machine learning model development.

The following points describe the structure and important characteristics of the Road Traffic Accident (RTA) data set used in this project:

• Data set Name: Road Traffic Accident (RTA) Data set
• Number of Rows: 12,316 accident records
• Number of Columns: 32 attributes/features
• Row Description: Each row represents one road traffic accident case
• Column Description: Each column contains information related to drivers, vehicles, road conditions, weather, and accident details
• Data Types: Contains both categorical and numerical data
• Numerical Features: Number_of_vehicles_involved, Number_of_casualties
• Categorical Features: Age_band_of_driver, Sex_of_driver, Driving_experience, Type_of_vehicle, Weather_conditions, Road_surface_conditions, Cause_of_accident, etc.
• Target Variable: Accident_severity
• Classes in Target Variable: Fatal Injury, Serious Injury, Slight Injury
• Purpose of Data set: Used to analyze accident patterns and predict accident severity using machine learning models.

1.2 Flow Diagram



2. Modules for Project Implementation

2.1 Modules Used(inbuilt/ custom modules used and the purpose)

3. Modules for Project Implementation

1. Pandas

• Used for data manipulation and analysis
Functions used:
• pd.read_csv() → to load the RTA dataset from CSV file
• df.head() → to display first few rows of dataset
• df.sample() → to display random records
• df.info() → to get dataset structure and data types
• df.describe() → to generate summary statistics
• df.isnull().sum() → to check missing values
• df.groupby() → to group accident severity data
• df.value_counts() → to check class distribution
• df.drop() → to remove unnecessary columns
• df.fillna() → to handle missing values

2. NumPy

• Used for numerical computations and array operations
Functions used:
• np.int64 → to represent integer values
• np.array() → for numerical array processing

3. Matplotlib

• Used for data visualization and plotting graphs
Functions used:
• plt.figure() → to create figure window
• plt.show() → to display plots
• plt.pie() → to create pie charts
• plt.hist() → to generate histograms

4. Seaborn

• Used for advanced statistical visualization
Functions used:
• sns.boxplot() → to create boxplots

• sns.scatterplot() → to create scatter plots
• sns.pairplot() → to visualize feature relationships
• sns.heatmap() → to display correlation matrix and confusion matrix
• sns.countplot() → to visualize categorical distributions
• sns.FacetGrid() → to create grouped visualizations

5. Scikit-learn (sklearn)

• Core library used for machine learning model building and evaluation

a) Model Selection

• train_test_split() → to split data into training and testing sets
• RepeatedStratifiedKFold() → for repeated cross-validation
• GridSearchCV() → for hyperparameter tuning
• KFold() → for fold-based validation

b) Preprocessing

• StandardScaler() → to standardize feature values
• MinMaxScaler() → to normalize feature values
• LabelEncoder() → to convert categorical data into numeric form

c) Pipeline

• Pipeline() → to combine preprocessing and classification steps

d) Machine Learning Algorithms

• LogisticRegression() → for logistic regression classification
• DecisionTreeClassifier() → for decision tree prediction
• SVC() → for support vector machine classification
• KNeighborsClassifier() → for KNN classification
• GaussianNB() → for Naive Bayes classification
• RandomForestClassifier() → for random forest classification
• AdaBoostClassifier() → for boosting-based classification
• GradientBoostingClassifier() → for gradient boosting classification

e) Evaluation Metrics

• accuracy_score() → to calculate prediction accuracy
• classification_report() → to generate precision, recall, and F1-score
• confusion_matrix() → to evaluate prediction results

6. Imbalanced-learn (imblearn)

• Used for handling imbalanced dataset classes
Functions used:
• SMOTE() → to oversample minority classes and balance the dataset

7. Collections

• Used for counting dataset class distribution
Functions used:
• Counter() → to count occurrences of accident severity classes

8. Warnings

• Used to suppress unnecessary warning messages
Functions used:
• warnings.filterwarnings('ignore') → to ignore warning outputs

2.2 Platform Used:

Jupyter Notebook

• Jupyter Notebook is used as the primary development platform for implementing the Road Traffic Accident Severity Prediction system.

Reason for using:
• Provides an interactive environment to write and execute machine learning code step-by-step
• Helps in data preprocessing, visualization, model training, and evaluation in a single workspace
• Allows easy visualization of graphs such as histograms, boxplots, heatmaps, scatter plots, and confusion matrix
• Useful for testing multiple machine learning algorithms like Logistic Regression, Decision Tree, SVM, KNN, Random Forest, AdaBoost, and Gradient Boosting incrementally
• Supports integration of important libraries such as Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, and Imbalanced-learn
• Makes debugging, modifying, and comparing models easier during experimentation
• Enables displaying dataset outputs, accuracy scores, and classification reports directly alongside the code

3.Machine Learning Model/ Searching Technique Used:

This project uses multiple machine learning algorithms to predict the severity of road traffic accidents based on accident-related factors such as driver information, road conditions, weather conditions, number of vehicles involved, and number of casualties. The models are trained on preprocessed and balanced data using feature encoding, scaling, and SMOTE oversampling techniques.

How it works:
• Assumes feature independence
• Calculates probability for each class
• Predicts the class with highest probability

Why used:
• Fast and computationally efficient
• Works well with large datasets
• Suitable for probabilistic classification

1. Random Forest Classifier

Description:
Random Forest is an ensemble learning algorithm that combines multiple decision trees for better prediction accuracy.

How it works:
• Creates multiple decision trees using random subsets of data
• Combines outputs from all trees
• Predicts the final class using majority voting

Why used:
• Reduces overfitting problem
• Provides higher accuracy and better performance
• Achieved the best accuracy in this project

1. AdaBoost Classifier

Description:
AdaBoost is a boosting algorithm that improves prediction performance by combining weak learners.

How it works:
• Trains multiple weak classifiers sequentially
• Gives more importance to wrongly classified samples
• Combines all weak learners into a strong classifier

Why used:
• Improves model performance
• Reduces classification errors
• Enhances prediction accuracy

1. Gradient Boosting Classifier

Description:
Gradient Boosting is an ensemble learning technique that builds models sequentially to reduce prediction errors.

How it works:
• Builds weak models one after another
• Minimizes previous model errors using gradient optimization
• Produces strong predictive performance

Why used:
• Provides high predictive accuracy
• Handles complex datasets efficiently
• Improves overall model performance

1. SMOTE (Synthetic Minority Oversampling Technique)

Description:
SMOTE is used to handle class imbalance in the dataset.

How it works:
• Generates synthetic samples for minority classes
• Balances the dataset distribution
• Improves model training on imbalanced data

Why used:
• Prevents biased prediction toward majority class
• Improves classification performance
• Enhances prediction accuracy for minority classes

1. GridSearchCV

Description:
GridSearchCV is used for hyperparameter tuning and selecting the best model parameters.

How it works:
• Tests multiple parameter combinations
• Uses cross-validation for evaluation
• Selects the best-performing parameter set

Why used:
• Optimizes model performance
• Improves prediction accuracy
• Helps select best hyperparameters automatically

4. Evaluation metrics used:

5. Evaluation Metrics Used

(Describe each evaluation metric used along with formulae)

1. Confusion Matrix

Description:
A confusion matrix is a table used to evaluate the performance of a classification model by comparing actual and predicted accident severity classes.

Predicted Fatal/Serious/Slight
Actual Class Correct / Incorrect Prediction

Where:
• TP (True Positive): Accident severity correctly predicted
• TN (True Negative): Correct non-target class prediction
• FP (False Positive): Incorrectly predicted as target class
• FN (False Negative): Failed to predict the target class correctly

Why used:
• Helps visualize model prediction performance
• Shows classification errors clearly
• Useful for multi-class accident severity analysis

---

2. Accuracy

Description:
Accuracy measures the overall correctness of the machine learning model.

Formula:

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Interpretation:
• Higher accuracy indicates better overall prediction performance
• In this project, Random Forest achieved the highest accuracy among all models

Why used:
• Simple and widely used evaluation metric
• Measures total correct predictions made by the model

---

3. Precision

Description:
Precision measures how many predicted accident severity cases are actually correct.

Formula:

Precision = TP / (TP + FP)

Interpretation:
• High precision means fewer false positive predictions
• Indicates reliability of positive predictions

Why used:
• Helps evaluate correctness of predicted accident classes
• Important when false predictions must be minimized

---

4. Recall

Description:
Recall measures how many actual positive cases are correctly identified by the model.

Formula:

Recall = TP / (TP + FN)

Interpretation:
• High recall means fewer missed accident severity cases
• Indicates model's ability to detect actual positive cases

Why used:
• Important for identifying severe accident cases correctly
• Helps reduce false negatives

---

5. F1-Score

Description:
F1-Score is the harmonic mean of Precision and Recall.

Formula:

F1-Score = (2 × Precision × Recall) / (Precision + Recall)

Interpretation:
• Higher F1-score indicates balanced model performance
• Useful when dataset classes are imbalanced

Why used:
• Combines both precision and recall into a single metric
• Provides balanced evaluation of the model

---

6. Classification Report

Description:
Classification Report summarizes Precision, Recall, F1-Score, and Support values for each class.

Contents:
• Precision
• Recall
• F1-Score
• Support (number of actual samples)

Why used:
• Provides detailed performance analysis for all accident severity classes
• Helps compare model performance across categories

---

7. Cross-Validation Score

Description:
Cross-validation is used to evaluate model performance on multiple subsets of the dataset.

How it works:
• Splits dataset into multiple folds
• Trains and tests model repeatedly
• Computes average performance score

Why used:
• Reduces overfitting
• Provides reliable model evaluation
• Ensures model generalization capability

---

8. GridSearchCV Evaluation

Description:
GridSearchCV evaluates multiple parameter combinations to select the best-performing model.

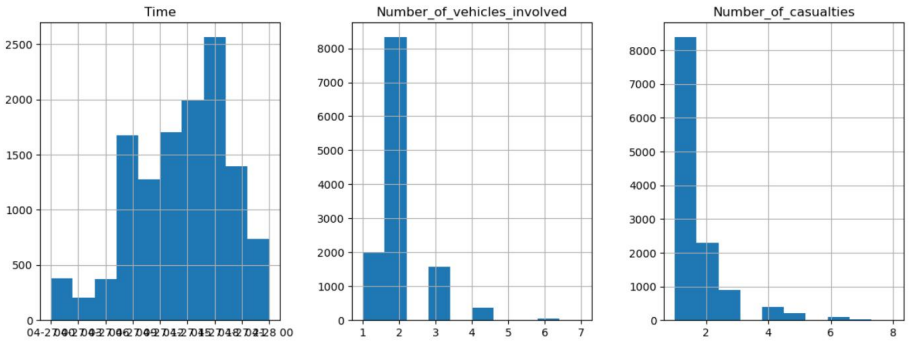
How it works:
• Performs hyperparameter tuning
• Uses cross-validation internally
• Selects best parameter combination automatically

Why used:
• Improves model accuracy
• Optimizes classifier performance
• Helps identify best hyperparameters automatically

5. Results and Visualizations

```
Sex_of_driver                  0
Educational_level            741
Vehicle_driver_relation      579
Driving_experience           829
Type_of_vehicle              950
Owner_of_vehicle             482
Service_year_of_vehicle     3928
Defect_of_vehicle           4427
Area_accident_occured        239
Lanes_or_Medians             385
Road_allignment              142
Types_of_Junction            887
Road_surface_type            172
Road_surface_conditions        0
Light_conditions               0
Weather_conditions             0
Type_of_collision            155
Number_of_vehicles_involved    0
Number_of_casualties           0
Vehicle_movement             308
Casualty_class                 0
Sex_of_casualty                0
Age_band_of_casualty           0
Casualty_severity              0
Work_of_casuality           3198
Fitness_of_casuality        2635
Pedestrian_movement            0
Cause_of_accident              0
Accident_severity              0
dtype: int64
```

In [16]:
```
df.hist(layout=(1,6), figsize=(30,5))
plt.show()
```



In [17]:
```
df['Number_of_casualties'].value_counts()
```

Fig. 1: Histogram Analysis of Time, Number of Vehicles Involved, and Number of Casualties

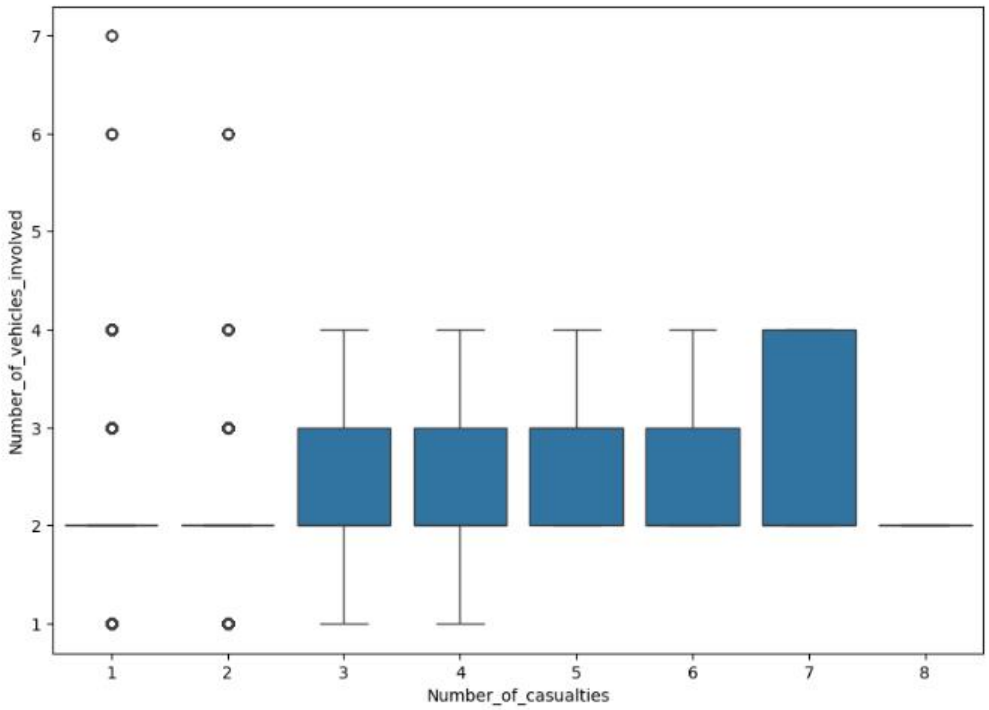


Fig 2: Box Plot Analysis of Number of Casualties vs Number of Vehicles Involved

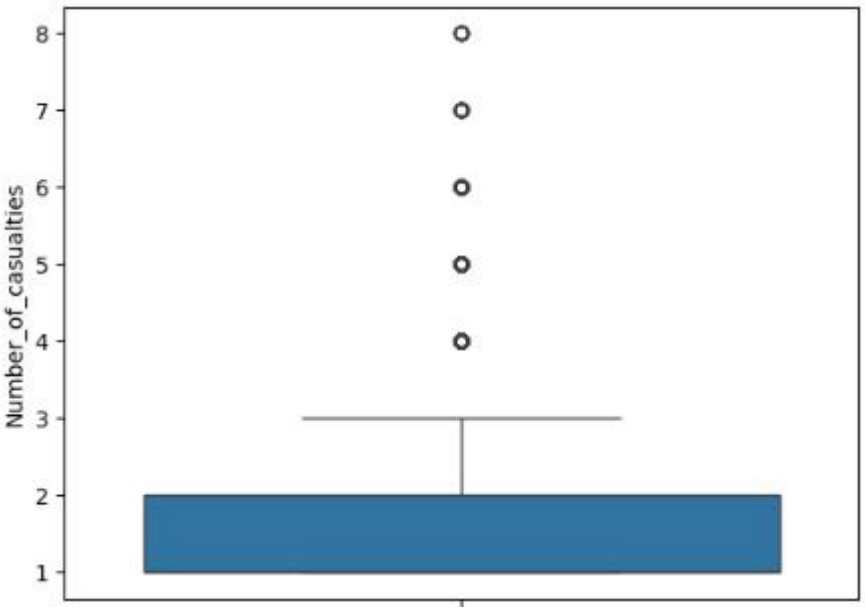


Fig 3: Box Plot Analysis of Number of Casualties

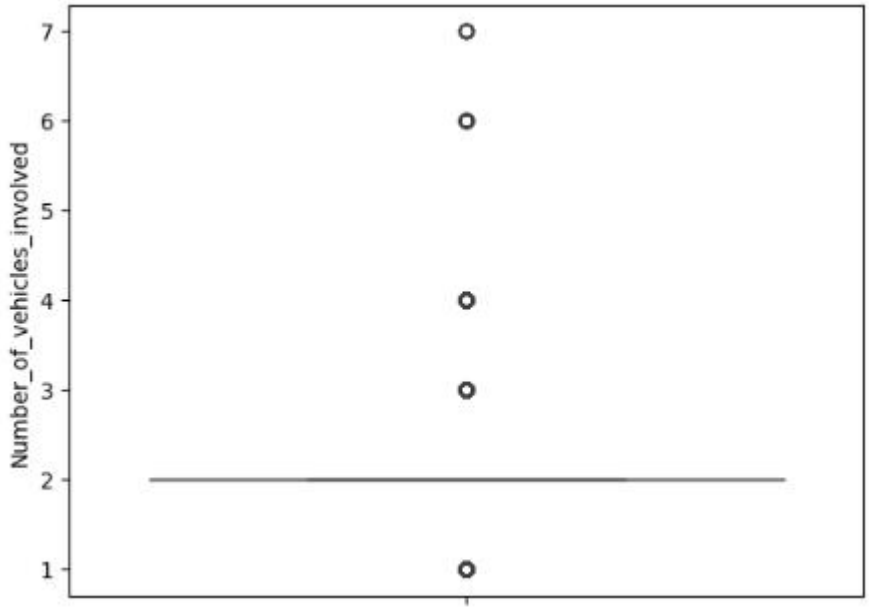


Fig 4: Box Plot Analysis of Number of Vehicles Involved

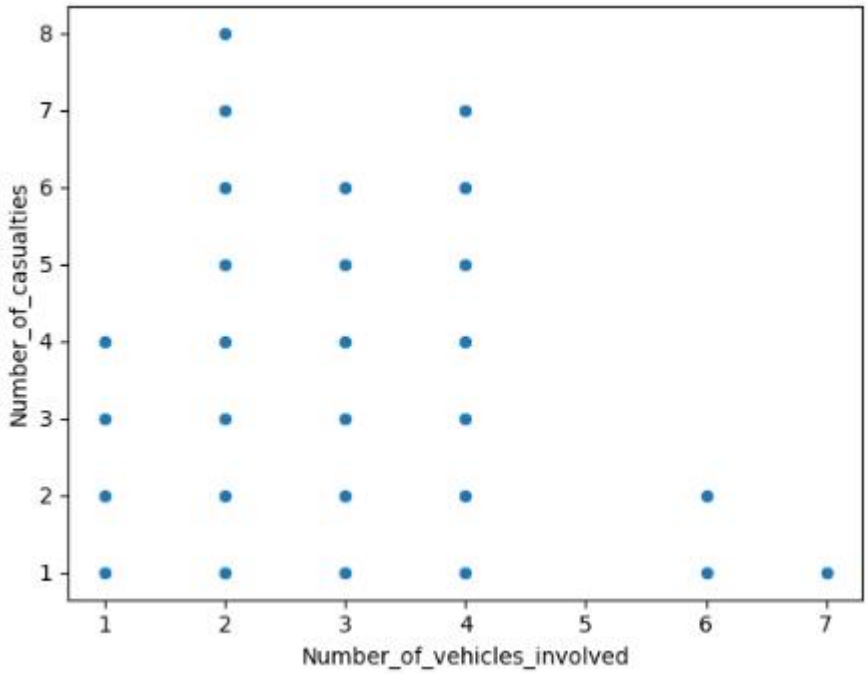


Fig 5: Scatter Plot of Number of Vehicles Involved vs Number of Casualties

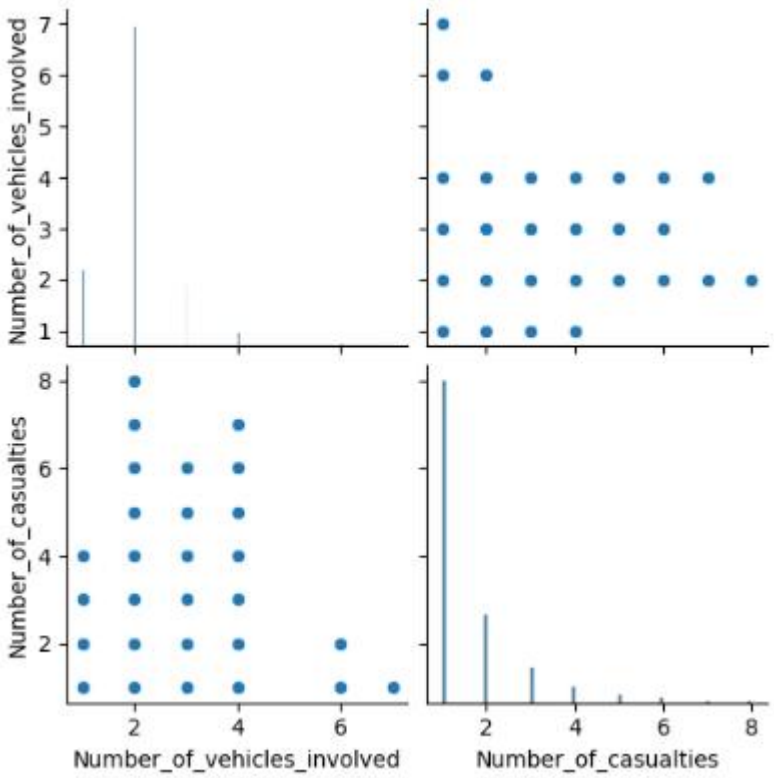


Fig 6: Pair Plot Analysis of Number of Vehicles Involved and Number of Casualties

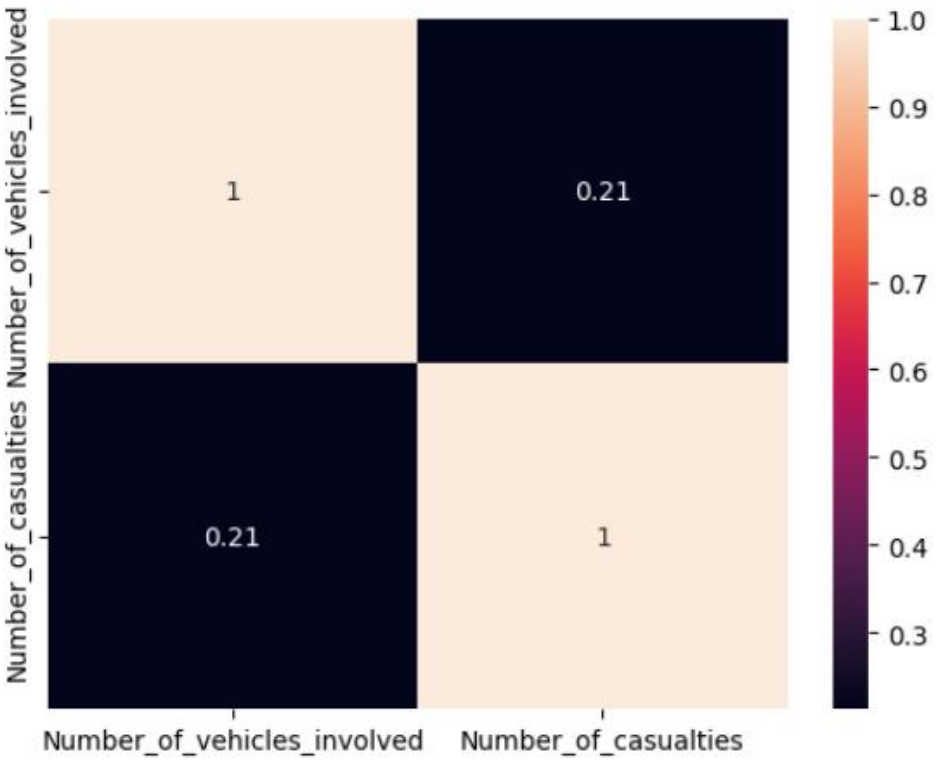


Fig 7: Correlation Heatmap of Number of Vehicles Involved and Number of Casualties

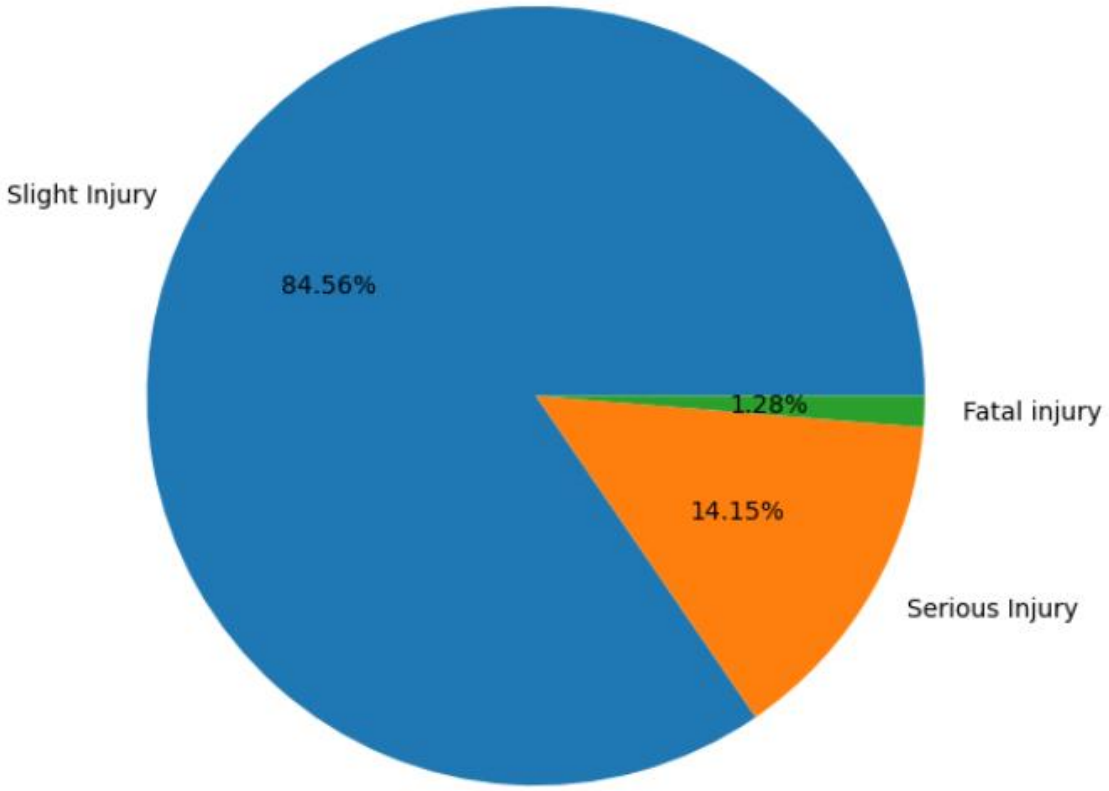


Fig 8 Pie Chart of Accident Severity Distribution

:

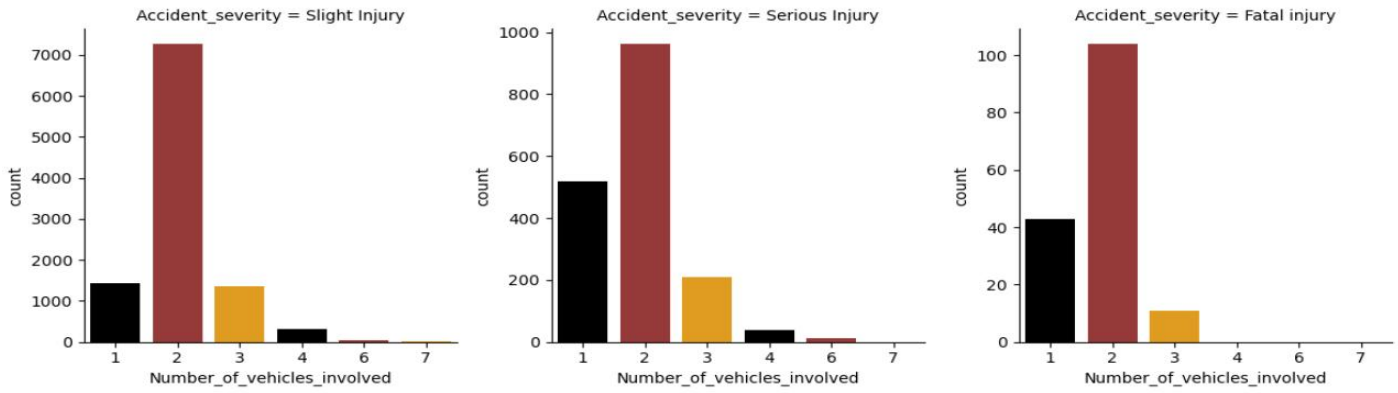


Fig 9: Facet Grid Analysis of Accident Severity vs Number of Vehicles Involved

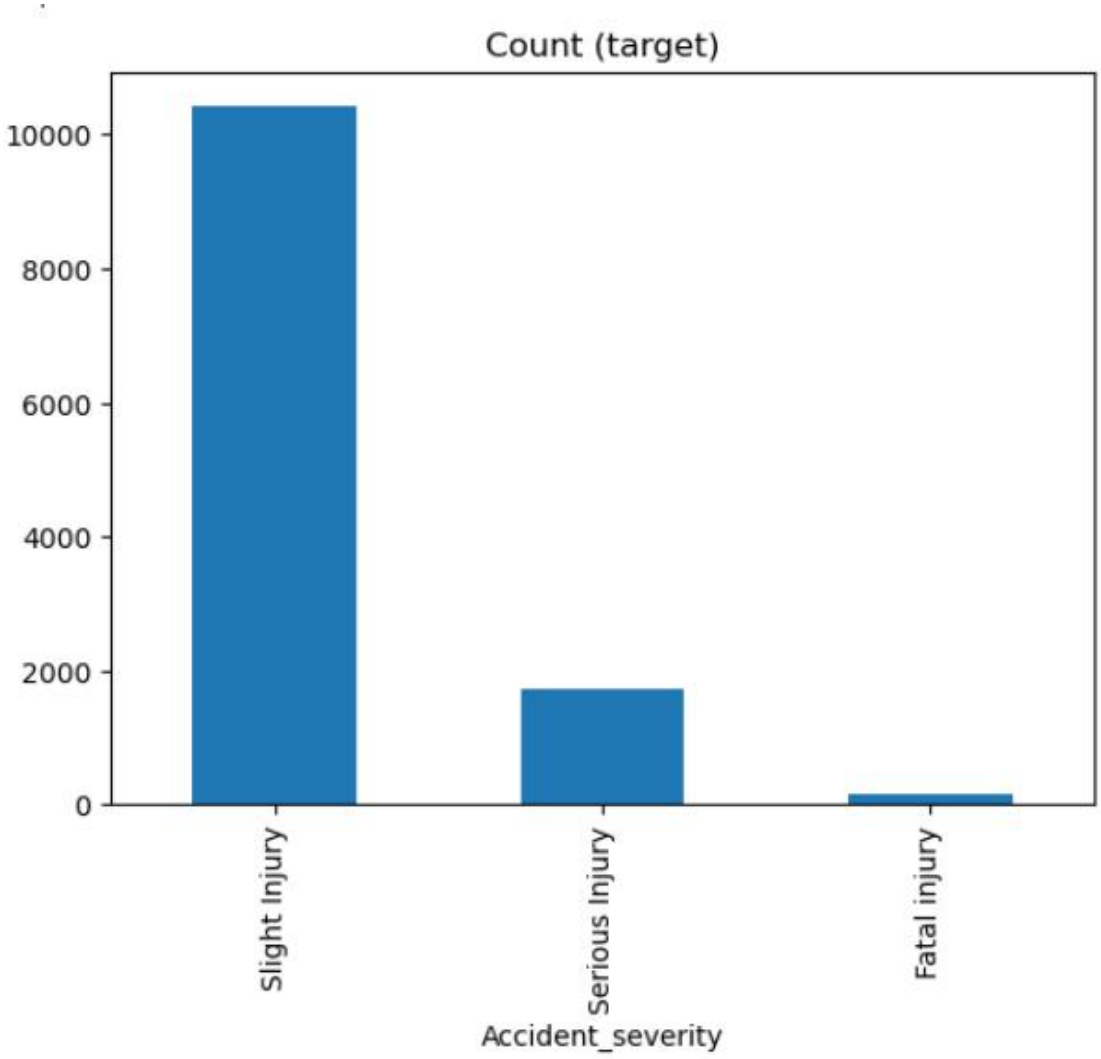


Fig 10: Bar Plot of Accident Severity Class Distribution

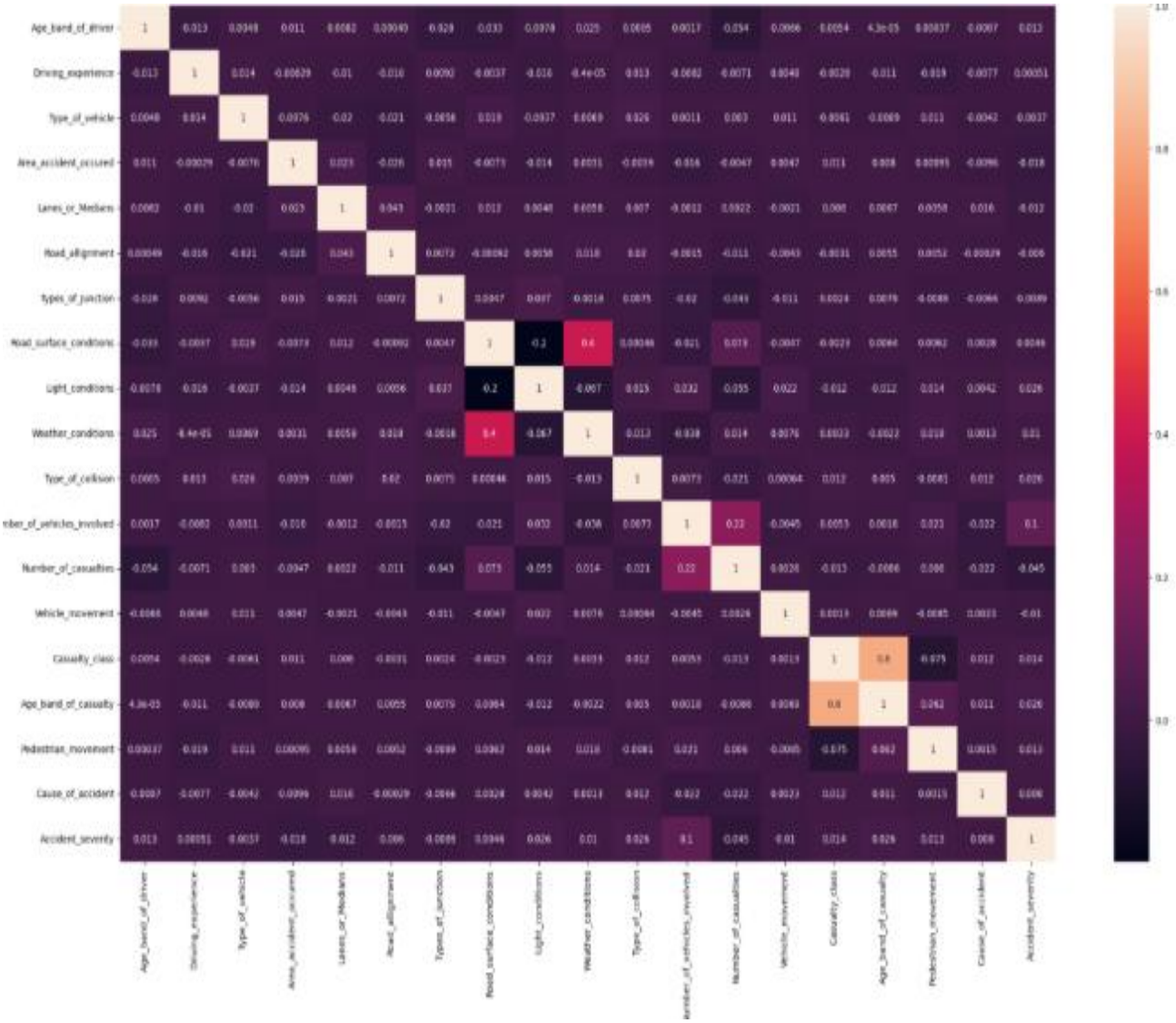


Fig 11: Correlation Heatmap of Road Traffic Accident Dataset Features

```
0.7718538565629228
              precision    recall  f1-score   support

           0       0.04      0.02      0.03        52
           1       0.24      0.19      0.22       552
           2       0.85      0.89      0.87      3091

    accuracy                           0.77      3695
   macro avg       0.38      0.37      0.37      3695
weighted avg       0.75      0.77      0.76      3695
```

3]: <Axes: >

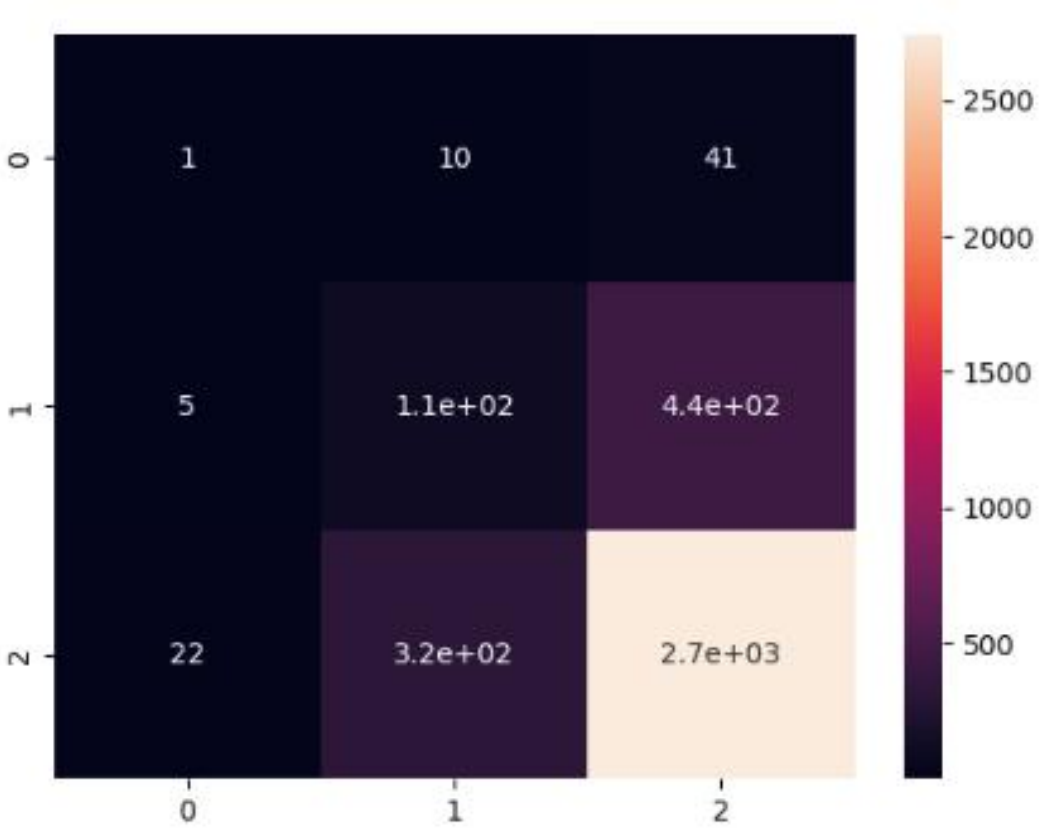


Fig 12: Confusion Matrix Heatmap for Accident Severity Prediction

6. Conclusion and Future Scope:

Conclusion

In this project, a Road Traffic Accident Severity Prediction system was successfully developed using machine learning techniques. The system processes accident-related data, performs preprocessing and feature engineering, and predicts accident severity categories such as Fatal Injury, Serious Injury, and Slight Injury.

Multiple machine learning models were implemented and compared, including:
• Logistic Regression
• Decision Tree Classifier
• Support Vector Machine (SVM)
• K-Nearest Neighbors (KNN)
• Gaussian Naive Bayes (GNB)
• Random Forest Classifier
• AdaBoost Classifier
• Gradient Boosting Classifier

The dataset was preprocessed using techniques such as handling missing values, label encoding, feature scaling, and SMOTE oversampling to improve model performance and handle class imbalance.

The performance of these models was evaluated using accuracy score, precision, recall, F1-score, classification report, and confusion matrix. The results showed that:
• Random Forest Classifier achieved the highest accuracy among all implemented models
• The model successfully classified most Slight Injury cases with good prediction performance
• SMOTE improved the handling of minority classes such as Fatal Injury and Serious Injury
• The system demonstrated effective prediction capability for accident severity analysis

Overall, the project demonstrates the usefulness of machine learning techniques in analyzing road traffic accident data and predicting accident severity for improving transportation safety and decision-making.

Future Scope

• Use advanced deep learning models for better prediction accuracy
• Implement real-time accident severity prediction systems
• Integrate GPS and live traffic data for enhanced analysis
• Improve prediction performance for minority classes like Fatal Injury

• Develop a web-based or mobile application for accident prediction
• Use larger and real-time datasets for better generalization
• Apply feature selection techniques to improve model efficiency

7. References

Books:

1. Python for Data Analysis – Wes McKinney
2. Hands-On Machine Learning with Scikit-Learn and TensorFlow – Aurélien Géron
3. Machine Learning using Python – Manaranjan Pradhan
4. Data Science from Scratch – Joel Grus

Online Resources:

• GeeksforGeeks – https://www.geeksforgeeks.org/
• TutorialsPoint – https://www.tutorialspoint.com/
• Scikit-learn Documentation – https://scikit-learn.org/
• Pandas Documentation – https://pandas.pydata.org/
• NumPy Documentation – https://numpy.org/
• Matplotlib Documentation – https://matplotlib.org/
• Seaborn Documentation – https://seaborn.pydata.org/
• Kaggle Datasets – https://www.kaggle.com/
• Stack Overflow – https://stackoverflow.com/